USER GUIDE · V1.0

open-scraper

Two read-only scrapers that work on any public website

page_scraper.py grabs pages right now and saves them as clean Markdown. **daemon.py** keeps watching a list of pages and saves a copy every time something actually changes.

No accounts. No API keys. No passwords. The tools only ever send plain `GET` requests — they cannot log in anywhere, by design.

INSTALL TIME
About 2 minutes, three commands

PYTHON
3.10 or newer

OUTPUT
Markdown files in `output/`, never uploaded

DEPENDENCIES
Playwright only

LICENSE
MIT — use it, fork it, ship it

POLITENESS
robots.txt honoured, 2-second pause between pages

```
https://github.com/arushvantore/open-scraper
```

Created by **@arushvantore** · Follow this guide top to bottom and you will have both tools running without needing any further help.

1 Install

One time only. Takes about two minutes.

You need **Python 3.10 or newer**. Check yours:

TERMINAL

```
$ python3 --version
$ Python 3.14.7
```

COPY AND PASTE THESE FOUR COMMANDS

TERMINAL

```
$ git clone https://github.com/arushvantore/open-scraper.git
$ cd open-scraper
$ python3 -m venv .venv
$ .venv/bin/pip install -r requirements.txt
$ .venv/bin/playwright install chromium
```

WHAT THESE DO

`git clone` copies the repo · `venv` makes a private Python sandbox for this project · `pip install` pulls in Playwright · `playwright install chromium` downloads the browser it drives.

WINDOWS

Use `.venv\Scripts\python` everywhere this guide writes `.venv/bin/python`. The rest is identical.

CHECK IT WORKED

TERMINAL

```
$ .venv/bin/python page_scraper.py https://example.com
```

YOU SHOULD SEE

```
open-scraper :: 1 page(s) -> output/
  GET   https://example.com
  SAVE  output/example-com.md  (133 words, hash a4ccdd75cf70ea93)

Done. 1 ok, 0 skipped/failed.
```

If you see `SAVE output/example-com.md`, installation is finished. If you see an error, jump to **Troubleshooting** on the last page.

From here on, every command starts with `cd open-scraper` (or `cd ~/open-scraper` if you cloned it into your home folder).

2 Scrape a single page

The most common thing you will do. One command, one file.

TERMINAL

```
$ cd open-scraper
$ .venv/bin/python page_scraper.py https://example.com
```

OUTPUT

```
open-scraper :: 1 page(s) -> output/
  GET   https://example.com
  SAVE  output/example-com.md  (133 words, hash a4ccdd75cf70ea93)

Done. 1 ok, 0 skipped/failed.
```

Open `output/example-com.md`. It looks like this:

OUTPUT/EXAMPLE-COM.MD

```
---
url: https://example.com/
title: Example Domain
fetched: 2026-10-27T19:51:56Z
selector: (whole page)
word_count: 133
content_hash: a4ccdd75cf70ea93
---

# Example Domain

This domain is for use in documentation examples without needing
permission. This is not a service; avoid relying on it for testing
and monitoring purposes.
```

> **WHAT SURVIVED THE CONVERSION**
> Headings, links, lists, quotes, code blocks and images all become clean Markdown. Scripts, styles and navigation menus are thrown away — so what you get is the actual content of the page.

SCRAPE SEVERAL PAGES AT ONCE

Just list them, separated by spaces:

TERMINAL

```
$ .venv/bin/python page_scraper.py https://a.com https://b.com https://c.com
```

It fetches them one at a time, with a 2-second pause between each, so you are never hammering a server.

3 Grab only part of a page

Use `--selector` when you want the article, not the whole site chrome.

Without a selector you get everything. With one, you get just that element:

TERMINAL

```
$ .venv/bin/python page_scraper.py https://www.python.org/downloads/ --selector "#content" --
links
```

OUTPUT

```
open-scraper :: 1 page(s) -> output/
  GET   https://www.python.org/downloads/
  SAVE  output/www-python-org-downloads.md  (2379 words, hash 2b14189d3b4d6968)

Done. 1 ok, 0 skipped/failed.
```

The file now starts with `selector: #content` in its header, and ends with a **Links** section listing every URL found inside it (because we passed `--links`).

HOW TO FIND A SELECTOR — 4 STEPS

1. Open the page in Chrome, Firefox or Edge.
2. **Right-click** the piece of text you want → **Inspect**.
3. In the panel that opens, **right-click** the highlighted line of HTML → **Copy** → **Copy selector**.
4. Paste it after `--selector`, wrapped in quotes.

EXAMPLE

```
$ .venv/bin/python page_scraper.py https://blog.example.com/post \
    --selector "article.post-body"
```

> **SELECTOR MATCHED NOTHING?**
> The tool stops and says so, rather than saving an empty file. Re-copy the selector, or drop `--selector` entirely to take the whole page.

THREE MORE FLAGS YOU WILL USE

FLAG	WHAT IT ADDS
`--screenshot`	A full-page `.png` saved next to the Markdown file.
`--wait 5000`	Waits 5 seconds for JavaScript to finish. Use on pages that render late.
`--json summary.json`	A machine-readable summary of everything you just scraped.

4 Scrape a list of pages from a file

For anything longer than a handful of URLs.

Edit the file called `targets.txt` that sits in the repo root. Paste one URL per line. Lines starting with `#` are ignored.

TARGETS.TXT

```
# One URL per line. Lines starting with # are ignored.
# Only public pages here - this tool cannot log in anywhere.

https://example.com
https://www.python.org
https://news.ycombinator.com
```

> **THREE RULES**
> Every URL must start with `https://` · one URL per line · no quotes or commas needed. Anything else and the tool stops with a clear message telling you which line is wrong.

Then run it:

TERMINAL

```
$ .venv/bin/python page_scraper.py --targets targets.txt
```

REAL OUTPUT

```
open-scraper :: 3 page(s) -> output/
  GET   https://example.com
  SAVE  output/example-com.md  (133 words, hash a4ccdd75cf70ea93)
  GET   https://www.python.org
  SAVE  output/www-python-org.md  (1409 words, hash c4779ed592c5605c)
  GET   https://news.ycombinator.com
  SAVE  output/news-ycombinator-com.md  (598 words, hash 1bf66fb2282146f5)

Done. 3 ok, 0 skipped/failed.
```

Three files are now in `output/`. Use `--out mydir` to send them somewhere else, and `--pause 5` to slow down between pages.

> **SAME FILE WORKS FOR BOTH TOOLS**
> `targets.txt` is read by **PAGE_SCRAPER.PY** and by **DAEMON.PY**. Maintain one list, drive both tools from it.

5 Watch pages for changes (the daemon)

Re-checks your list on a timer and archives only what actually changed.

TERMINAL — RUN FOREVER, CHECKING EVERY 5 MINUTES

```
$ .venv/bin/python daemon.py --targets targets.txt --interval 300
```

TERMINAL — RUN A SINGLE CHECK AND EXIT (GOOD FOR SCHEDULERS)

```
$ .venv/bin/python daemon.py --targets targets.txt --once
```

Press Ctrl-C to stop. It finishes the page it is on, saves its state, prints `daemon stopped`, and exits cleanly.

FIRST RUN — EVERYTHING IS NEW

```
2026-10-27T20:46:16Z daemon start: 3 target(s), interval 300s, out=output
  NEW   https://example.com -> 20261027_204620_example-com.md
2026-10-27T20:46:20Z CHANGED https://example.com (133 words)
  NEW   https://www.python.org -> 20261027_204625_www-python-org.md
2026-10-27T20:46:25Z CHANGED https://www.python.org (1409 words)
  NEW   https://news.ycombinator.com -> 20261027_204631_news-ycombinator-com.md
2026-10-27T20:46:31Z CHANGED https://news.ycombinator.com (598 words)
2026-10-27T20:46:33Z cycle done: 3 seen, 3 changed, 0 failed
```

SECOND RUN, SECONDS LATER — NOTHING MOVED

```
2026-10-27T20:46:33Z daemon start: 3 target(s), interval 300s, out=output
  same  https://example.com
2026-10-27T20:46:36Z same   https://example.com (133 words)
  same  https://www.python.org
2026-10-27T20:46:41Z same   https://www.python.org (1409 words)
  same  https://news.ycombinator.com
2026-10-27T20:46:47Z same   https://news.ycombinator.com (598 words)
2026-10-27T20:46:49Z cycle done: 3 seen, 0 changed, 0 failed
```

Unchanged pages are **not** re-archived, so your folder never fills up with identical copies.

DAEMON FLAGS WORTH KNOWING

FLAG	DOES
`--interval N`	Seconds between checks. Default `300` (5 min).
`--once`	One check, then exit.
`--diff-only`	Log changes but write no archive files.
`--selector "css"`	Watch one element instead of the whole page.
`--log FILE`	Where the timeline goes. Default `daemon.log`.

6 Where your files go

Everything lands in the `output/` folder — on your machine only.

```
output/
├── live/       current state of each watched page  (daemon)
├── archive/    one file per real change, stamped   (daemon)
│               e.g. 20261027_204620_example-com.md
├── *.md        one-shot scrapes                (page_scraper)
├── *.png       screenshots, if you used --screenshot
└── .state.json hashes the daemon uses to spot change
```

> **IMPORTANT — THIS FOLDER IS GITIGNORED**
>
> `output/` is listed in `.gitignore`, so scraped content can never be pushed to GitHub by accident. Your captures stay on your computer. The same is true of `daemon.log` and the `.venv/` folder.

EVERY FLAG IN ONE PLACE

FLAG	TOOL	WHAT IT DOES
`--targets FILE`	both	Read URLs from a file.
`--selector CSS`	both	Take only that element.
`--links`	scraper	Append every link found.
`--screenshot`	scraper	Also save a full-page PNG.
`--json FILE`	scraper	Write a JSON summary.
`--out DIR`	both	Choose the output folder.
`--timeout N`	both	Seconds to wait for a page. Default 30.
`--wait MS`	scraper	Extra time for JavaScript. Default 1500.
`--pause N`	both	Seconds between URLs. Default 2.
`--headful`	scraper	Show the browser window.
`--interval N`	daemon	Seconds between cycles. Default 300.
`--once`	daemon	Run one cycle, then exit.
`--diff-only`	daemon	Skip archive files.
`--no-robots`	both	Ignore robots.txt — **only on sites you own**.

Every tool also has `--help`, which prints this same list in your terminal.

7 Keep the daemon running after you close the terminal

Optional. Only needed if you want it running unattended.

By default the daemon lives only while its terminal window is open. To make it start on login and survive reboots, hand it to your operating system.

MACOS — LAUNCHD

TERMINAL

```
$ mkdir -p ~/Library/LaunchAgents
$ cat > ~/Library/LaunchAgents/com.example.scraper.plist <<'XML'
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0"><dict>
  <key>Label</key><string>com.example.scraper</string>
  <key>ProgramArguments</key><array>
    <string>/ABSOLUTE/PATH/open-scraper/.venv/bin/python</string>
    <string>/ABSOLUTE/PATH/open-scraper/daemon.py</string>
    <string>--targets</string>
    <string>/ABSOLUTE/PATH/open-scraper/targets.txt</string>
    <string>--interval</string><string>300</string>
  </array>
  <key>WorkingDirectory</key>
  <string>/ABSOLUTE/PATH/open-scraper</string>
  <key>RunAtLoad</key><true/>
  <key>KeepAlive</key><true/>
</dict></plist>
XML
$ launchctl load ~/Library/LaunchAgents/com.example.scraper.plist
```

> **REPLACE EVERY `/ABSOLUTE/PATH`**
> with the real location, e.g. `/Users/you/open-scraper`. Then stop it with `launchctl unload ~/Library/LaunchAgents/com.example.scraper.plist`.

LINUX — CRON, EVERY 10 MINUTES

RUN `CRONTAB -E` AND ADD THIS LINE

```
*/10 * * * * cd /path/open-scraper && .venv/bin/python daemon.py --targets targets.txt --once >> daemon.log 2>&1
```

WINDOWS — TASK SCHEDULER

Create a task that runs `C:\path\open-scraper\.venv\Scripts\python.exe daemon.py --targets targets.txt --once`, starting in `C:\path\open-scraper`, and tick *"Run whether user is logged on or not"*.

A ready-to-edit copy of the launchd file and cron line also lives in `README.md` in the repository.

8 Troubleshooting & the rules

What goes wrong, what it means, and how to fix it.

MESSAGE YOU SEE	WHAT IT MEANS / FIX
`Playwright is not installed`	Run `.venv/bin/pip install -r requirements.txt`.
Browser / executable errors	Run `.venv/bin/playwright install chromium`.
`Skip … (robots.txt disallows)`	Expected — that site asked not to be scraped. Add `--no-robots` only if you own it.
`Timeout 30000ms exceeded`	Slow site. Raise it: `--timeout 60`.
Output is empty	The page renders with JavaScript. Try `--wait 5000`, or add `--selector`.
`Selector … matched nothing`	Re-copy the CSS selector from your browser's DevTools.
`Targets file not found`	Wrong path, or you are not in the repo folder. Run `cd open-scraper` first, or pass URLs directly instead.
`these are not http(s) URLs`	A line in `targets.txt` is missing `https://`.
Daemon stops when I close the terminal	Normal — see section 7 to have the OS start it for you.

THE RULES THESE TOOLS FOLLOW

READ-ONLY, ALWAYS

They send `GET` requests and nothing else. They never submit a form, never post, never like, follow or upload. There is no password, cookie or session handling anywhere in the code — if a page is behind a login, these tools cannot open it.

ROBOTS.TXT IS RESPECTED BY DEFAULT

If a site's robots file says "do not fetch this", the tool skips it and prints `SKIP`. You have to ask for `--no-robots` explicitly.

YOUR RESPONSIBILITY

Only scrape pages you are allowed to scrape, and follow each site's Terms of Service. Politeness defaults (one page at a time, 2-second pauses) are there for a reason — use `--pause` to slow down further on small sites.

NOTHING PERSONAL IS IN THIS REPOSITORY

No credentials, no browser profiles, no scraped data. Everything you capture goes into `output/`, which is gitignored.

THAT'S IT

You now have both tools installed, tested, and understood. The full reference is `README.md` in the repository, and this same guide is kept beside it.

```
https://github.com/arushvantore/open-scraper
```

---

open-scraper · MIT License · Created by **@arushvantore** · Guide generated from a real, working install.